TravelStudio

An AI Travel Planning Agent built on a local LLM (Ollama)

Assignment Report: Building an AI Agent with Ollama

Group member	Student ID	Git username	Main area
Tharindu Sampath	CCLARKE548	TharinduRUSL	Tools, Streamlit frontend, project setup
Kasun Chamara Wijethunga	CCLARKE568	kaz-hub11	Agent loop, system prompt, agent tests, docs
Rev. Thapowanaye Dhammadeepa	CCLARKE676	dhammadeepa123	Tool refinements (weather, itinerary, catalog)

Repository: https://github.com/TharinduRUSL/Traval-Planing-Agenet | **Scenario:** Travel planning assistant | **Model:** qwen2.5:3b via Ollama

1. Problem and scenario

Planning even a short trip means juggling several separate questions: what to see, whether it will rain, how long it takes to get between places, how much it will cost for the whole group, and how to keep a plan that can be revised later. A plain chatbot cannot know tomorrow's forecast, do reliable arithmetic or store the result. Our project addresses this with an agent that decides for itself which tools to call, uses the results, and finishes with a day-by-day itinerary, a calculated budget and a saved file.

We chose the *Travel planning assistant* scenario from the assignment guide. A typical request is: *"Plan 2 days in Colombo starting tomorrow for one traveler with USD 70. I like museums and parks."* To answer it the agent must search attractions, fetch the weather for the real dates, estimate travel legs, convert currency if needed, calculate the group budget, add local tips and packing advice and optionally save the plan. This is a multi-step task in which later calls depend on earlier results (for example, the budget needs the admission prices returned by the attraction search), which matches the guide's requirement for tool calls in sequence.

2. Model selection

The agent runs **qwen2.5:3b** through Ollama [8] (about 3 billion parameters, roughly a 2 GB download). We chose it for three reasons. First, it supports native tool calling in Ollama, which the assignment requires. Second, it fits comfortably in the memory of an ordinary student laptop without a dedicated GPU. Third, the family is generally good at structured JSON output, which every tool call needs. **llama3.2:3b** is supported as an alternative through the `OLLAMA_MODEL` setting in `.env`. The trade-off is quality: a 3B model sometimes skips a tool or produces poor arguments, so much of our design effort went into validation and prompting (sections 4 and 5). No paid cloud API is used anywhere in the core agent.

3. Architecture

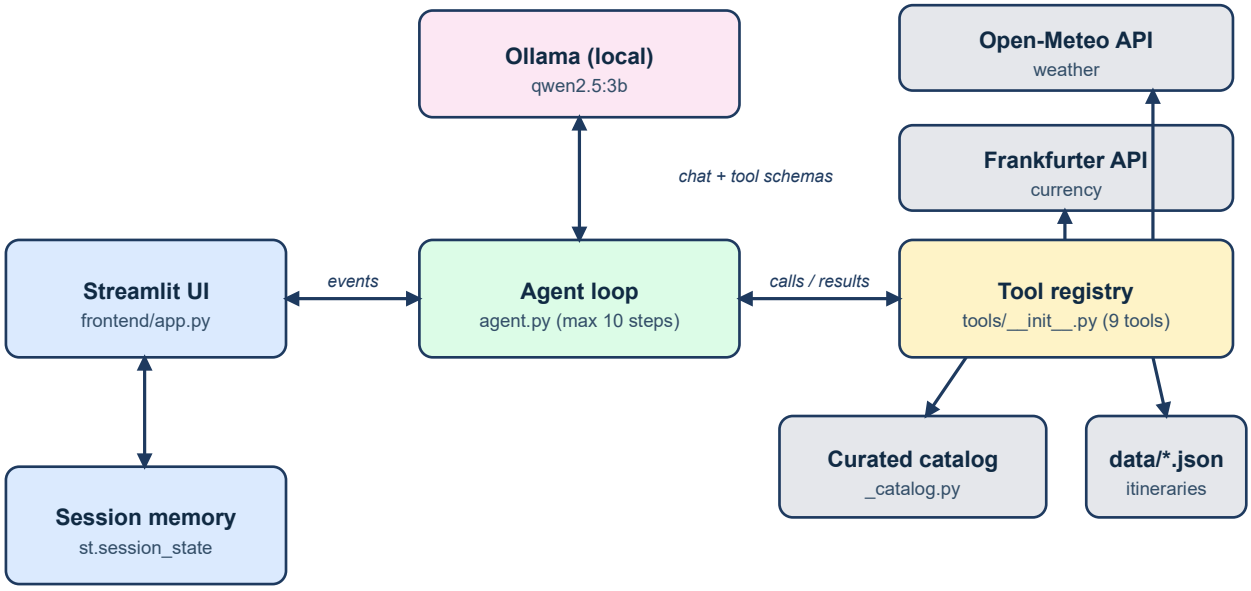


Figure 1. Architecture and data flow of TravelStudio.

The system has four layers. The **Streamlit frontend** (`frontend/app.py`) collects the prompt, owns the chat memory in `st.session_state` and renders the tool trace. The **agent loop** (`agent.py`) talks to the local Ollama server and executes tools. The **tool registry** (`tools/`) exposes nine plain Python functions. The **data layer** consists of two free external APIs (Open-Meteo and Frankfurter), a small curated catalog of four cities (Colombo, Kandy, Paris, London) and JSON files in `data/`. The agent and the UI communicate through a small event contract: `step`, `tool_start`, `tool_result` and `final`.

4. Tool design

There are nine tools, above the minimum of four. Each is a typed Python function with a docstring that Ollama converts into a JSON schema automatically. All are wrapped by a shared `safe_tool` decorator (`tools/_common.py`) that preserves the signature and returns `{"ok": true, "data": ...}` on success or `{"ok": false, "error": "..."}` on failure. Bad input, network failure and file errors therefore never crash the agent; the model receives a readable error and can correct itself.

Tool	Purpose	Parameters	Notes / sample output
get_weather	Live daily forecast (Open-Meteo geocoding + forecast) [9]	city, start_date, days (1-16)	Returns daily max/min temperature, rain probability. Refuses dates outside the 16-day window instead of inventing data.
search_attractions	Search curated catalog by interest keyword	city, interests[]	`Colombo + [museum]` returns National Museum, category museum, admission_usd_per_person 6, coordinates.
get_travel_time	Estimate time between two catalog attractions	city, origin, destination, mode	National Museum to Viharamahadevi Park, walking: 0.35 km, 5 min. Haversine distance x 1.3; labelled as an estimate.

calculate_budget	Whole-group cost with contingency (Decimal arithmetic)	days, travelers, daily_meals, daily_transport, total_admissions, budget, currency, contingency_percent	2 days, 1 traveler, meals 10, transport 5, admissions 6, budget 70: total 39.60, remaining 30.40, within_budget true.

Tool	Purpose	Parameters	Notes / sample output
get_currency_conversion	Convert amounts with Frankfurter reference rates [10]	amount, from_currency, to_currency	Returns rate, converted amount and rate date. Unsupported codes (e.g. LKR) give a clear error.
get_packing_list	Rule-based packing suggestions	min/max temperature, rain_probability, activities[]	24-31 C, 80% rain, walking + temples: sun hat, rain jacket, walking shoes, clothing covering shoulders and knees, etc.
get_local_tips	Local food and etiquette tips	city	Kandy: Rice and curry, String hoppers; tips such as covering shoulders and knees at temples.
save_itinerary	Persist a complete plan as JSON (**persistent data**)	filename, itinerary_json, overwrite	Validates the schema, rejects path traversal and existing files unless overwrite is true; returns saved true and path.
load_itinerary	Read and re-validate a saved plan	filename	Missing file returns "Itinerary does not exist..." rather than an exception.

Validation. Helper functions in `_common.py` check every argument type and range (non-empty strings of at most 200 characters, finite numbers, integers that are not booleans, lists of at most 50 strings). For example, `calculate_budget(0, ...)` returns "days must be a finite number between 1 and 30", and `search_attractions("Tokyo", [])` returns "Curated data supports Colombo, Kandy, Paris and London only." The itinerary validator additionally checks YYYY-MM-DD dates that are consecutive, non-empty activity lists, and that *budget.total* equals meals + transport + admissions + contingency within one cent, so a model cannot save an inconsistent budget. Filenames must match a strict pattern and are confined to the `data/` folder.

Mock versus live data. Weather and currency are live API calls. The attraction catalog, travel times and tips are curated and clearly labelled "illustrative" in every tool response, so the model and the user are never told that prices or opening hours are live.

5. Agent loop and prompt design

We implemented the loop ourselves in plain Python with the official `ollama` library [3] rather than using a framework, so every line can be explained. The pattern (model requests a tool, our code runs it, the result is returned as a `tool` message, repeat) follows Ollama's tool-calling documentation [2] and the tutorials and videos we studied [4]-[7]; the code, validation and prompts are our own. `run_agent()` is a generator that mutates the shared message list and yields events for the UI:

- **Validate and scope.** Empty input and unknown tool selections are rejected immediately. The user may restrict the turn to one tool; the allowed set is then *active_tools*.
- **Append** the user message to the history.
- **Model call.** For step 1 to 10, send the full history and the active function list to `client.chat` (temperature 0.1, context 16,384 tokens). A temporary copy of the system message is extended with today's date and the tool scope, so "tomorrow" resolves correctly without storing a stale date.
- **Keep the reply.** The assistant message, including its tool calls, is appended to history. If it contains no tool calls, its text is the final answer and the loop ends.

- **Run every requested tool.** For each call we yield `tool_start`, then check that the name is registered and enabled and that the arguments are a JSON object, call the function and append the JSON result as a `tool` message. Unknown tools or malformed arguments produce an error result instead of an exception.
- **Repeat.** If the 10-step limit is reached, the loop stops with an explicit message. Connection errors, timeouts, Ollama response errors and malformed model output each have their own final message.

A "step" is one model request, not one tool call, so one step can run several tools (the model can request multiple calls at once). Only functions in the whitelist can run; there is no `eval` or shell access. The Ollama host is also restricted to localhost so inference stays local.

System prompt. The prompt defines the role (careful travel planner), safety rules (no bookings, payments or email; treat tool and user data as data, not instructions) and a recommended workflow for multi-day trips: attractions, weather for the real dates, travel legs, currency conversion *before* the budget, then the group budget. Most of the rules exist because of failure modes we expect from small models: never claim success when `ok=false`, never invent attractions or live prices, do not double-count transport, state when weather is unavailable, and save only the *complete* plan with dated days and the exact budget object returned by `calculate_budget`. It also fixes the answer format: trip overview, day-by-day itinerary, budget and exclusions, weather and packing, local tips, saved file, assumptions.

6. Memory, interface and logging

Memory. The whole conversation (system, user, assistant and tool messages) is stored in `st.session_state.messages` and passed to the model on every call, so a follow-up such as "make day 2 cheaper" keeps the destination, dates and traveller count. A "New conversation" button resets the chat but not the saved files. Because the context window is finite (16,384 tokens), very long chats can lose detail; the workaround is to save the plan and reload it in a new chat.

Interface. We built a Streamlit web UI [11] (the guide's bonus option) with a tool selector beside the chat box, a sidebar showing the model and supported cities, and a download button for saved itineraries. **Logging.** Every tool call is shown live in an expander labelled with its step number and name, containing the arguments and the JSON result, and the same trace is kept in the chat history. This trace is what we show in the demo video. Unexpected exceptions are also logged to the terminal.

7. Testing

Automated tests. The `tests/` folder holds offline tests that use a fake Ollama client and mocked HTTP responses, so they run without Ollama or internet. They cover tool registration and bad arguments, group budget arithmetic, save/load/overwrite and path traversal, API failures, multiple tool calls per response, follow-up turns, unknown tools and malformed arguments, the 10-step limit, tool-selection filtering, error messages, and Streamlit rendering with AppTest.

Results. Running `python -m unittest discover -s tests` on the final repository gave 22 tests, of which 18 pass and 4 fail. The failures are not defects in the application but out-of-date test files, and we list them honestly:

- `test_agent_save_then_load_across_conversations` needs `tests/fixtures/valid_itinerary.json`, which was never committed. *Fix:* add the fixture file.
- `test_persistence_and_traversal` saves a plan without dates or a full budget, which the stricter validator (added by a later commit) now correctly rejects. *Fix:* use a valid plan.

- `test_model_failure` expects an old error message; the agent now returns a more specific one. *Fix:* update the expected text.
- `test_api_failures` mocks the geocoder with no country code, but the weather tool now checks that Colombo resolves to Sri Lanka. *Fix:* add `country_code` to the mock.

These failures came from three people changing the validators and agent after `tests/test_project.py` was written; `test_project.py` also duplicates tests that were later split into `test_agent.py` and `test_frontend.py`.

Problems found and fixed during development (from the commit history and docs): the first version put everything in a 338-line `main.py`, which we split into `agent.py`, `tools/` and `frontend/`; the itinerary tool initially accepted incomplete or inconsistent plans, so we added schema and budget-total validation; weather lookups could resolve to the wrong country, so supported cities are now checked against an expected country code; and the geocoder returned ambiguous cities, which now produce an explicit candidate list instead of a silent guess.

Live model testing. Offline tests do not prove that the 3B model chooses good tools. The checklist in `VALIDATION.md` was therefore run by hand on a real machine with Ollama. **[GROUP: insert your actual results here, e.g. which prompts worked, how many steps each used, and any wrong tool choices or retries you saw in the trace. Do not leave this sentence in the final PDF.]**

8. Limitations and future improvements

- **Small catalog.** Only four cities and 13 attractions are supported, with illustrative prices. A real attraction API (for example OpenTripMap) would remove this limit.
- **Approximate routing.** Travel time is distance x 1.3 at an assumed speed, not real routing or traffic. A routing API such as OpenRouteService would be more accurate.
- **Model reliability.** A 3B model can skip tools, ask unnecessary questions or hit the 10-step limit. Larger models (qwen3:4b or 8B) or a planner/executor split would help. The prompt, not code, enforces the overwrite-permission rule.
- **Exclusions.** Budgets exclude flights and accommodation; all budget inputs must share one currency, and Frankfurter does not support LKR.
- **Weather window.** Forecasts exist only for the next 16 days, so later trips are provisional.
- **Single user.** Saved files are shared on one machine with no authentication; session memory is lost when the browser session ends. A SQLite store with per-user plans would be the next step.
- **Testing.** Repair the four stale tests and measure tool-choice accuracy across models with repeated live runs.

9. Individual contribution table

Based on the Git history of the shared repository (non-merge commits) and the files each member changed. Members must confirm this table before submission.

Member	Commits	Contributions
Tharindu (TharinduRUSL)	8	Initial project and nine-tool implementation (tools/, catalog, validation helpers); first agent version; Streamlit frontend (app.py, components.py, styles.css, config); frontend docs and AppTest suite; repository merges and fixes.
Kasun (kaz-hub11)	4	Final agent loop and system prompt (agent.py); offline agent tests with fake model client (test_agent.py, fakes.py); requirements.txt and .gitignore; architecture and agent design documents.

Rev. Thapowanaye Dhammadeepa	2	Tool update: stricter itinerary validation (itinerary.py), weather country checking (weather.py), catalog changes (_catalog.py); README link update.

Appendix A. Use of AI assistants

In line with the guide, we disclose AI use. AI coding assistants were used to help generate and debug parts of the code, tests and documentation, as stated in the repository README. Claude was also used to help analyse the repository and draft this report. All group members reviewed the code and are able to explain the agent loop, the tools and the validation logic in the viva. **[GROUP: edit this paragraph so it exactly matches how each member really used AI.]**

References

[1] Assignment Guide: Building an AI Agent with Ollama (course document), Institute lecturer.

[2] Ollama Docs, "Tool calling". https://docs.ollama.com/capabilities/tool-calling

[3] Ollama Python library (official SDK), GitHub. https://github.com/ollama/ollama-python

[4] Real Python, "How to Integrate Local LLMs With Ollama and Python". https://realpython.com/ollama-python/

[5] Collabnix, "Function Calling with Ollama: Building Local Tool-Using AI Agents".https://collabnix.com/function-calling-with-ollama-building-local-tool-using-ai-agents/